

PRÁCTICA N.º 7 — HOJA DE EVIDENCIAS

Pruebas automatizadas de un microservicio

COM-600 · Microservicios · Gestión 2/2026 · Tema 5 — Desarrollo con Frameworks y Pruebas Automatizadas

Nombre y apellidos	Freddy Daniel Valencia Medina
Código universitario (CU)	35-5808
Carrera y paralelo	Ingeniería de Sistemas
Fecha de entrega	08/10/2026
Sistema operativo, versión de Node y de Docker	Linux(POP OS), node 22.23.2, Docker version 29.8.0
Repositorio (URL)	https://github.com/VMKayser/MicroserviciosFDVM/tree/main/laboratorios/07-pruebas-automatizadas/practica7-pruebas

Cómo se llena: Este documento se completa mientras se ejecuta la Parte 1 del enunciado. Cada captura y cada pregunta lleva el mismo número aquí y allá.

Las capturas deben ser de su propia máquina: debe verse el nombre de usuario o del equipo en la terminal. Recorte solo lo necesario, pero que el comando y su salida se lean con claridad.

Al terminar, exporte este archivo a PDF y súbalo a la plataforma junto con el enlace del repositorio.

Control de avance

Marque con una X a medida que complete cada laboratorio.

Laboratorio	Capturas	Preguntas	Completado
Laboratorio 0 — Verificación del entorno	1, 2	1	
Laboratorio 1 — La primera prueba unitaria	3	2	
Laboratorio 2 — Casos borde y errores: donde viven los defectos	4, 5	3	
Laboratorio 3 — Cobertura: qué mide y qué no	6, 7	4	
Laboratorio 4 — Probar el endpoint HTTP sin levantar el servidor	8, 9	—	
Laboratorio 5 — Integración real con un contenedor efímero	10, 11	5	
Laboratorio 6 — Prueba de contrato entre dos servicios	12, 13	6	
Laboratorio 7 — Dejar el proyecto listo para el pipeline	14, 15	7	

PARTE 1 — EVIDENCIAS DE LOS LABORATORIOS

Laboratorio 0 — Verificación del entorno

Captura N.º 1: salida de los cuatro comandos anteriores en su terminal.

```
kayser@pop-os:~/Documents/NEXUS/UNIVERSIDAD/Octavo_Semestre/com600_microservicios/proyecto$ cd ..
kayser@pop-os:~/Documents/NEXUS/UNIVERSIDAD/Octavo_Semestre/com600_microservicios$ node --version
v22.23.2
kayser@pop-os:~/Documents/NEXUS/UNIVERSIDAD/Octavo_Semestre/com600_microservicios$ npm --version
10.9.8
kayser@pop-os:~/Documents/NEXUS/UNIVERSIDAD/Octavo_Semestre/com600_microservicios$ docker --version
Docker version 29.0.2, build 7fc2dff
kayser@pop-os:~/Documents/NEXUS/UNIVERSIDAD/Octavo_Semestre/com600_microservicios$ docker ps
CONTAINER ID   IMAGE                  COMMAND                  CREATED        STATUS        PORTS                               NAMES
fb8ba4ae19bd   moodle-sandbox-moodle-sandbox-app   "httpd -D FOREGROUND"   7 days ago    Up 6 hours   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   moodle_sandbox_web
bcbdf3d2ee1c   postgres:17-alpine    "docker-entrypoint.s..." 6 weeks ago   Up 6 hours (healthy)   0.0.0.0:5433->5432/tcp, [::]:5433->5432/tcp   bd_postgres
```

Pregunta N.º 1. ¿Por qué esta práctica necesita Docker si lo que vamos a escribir son pruebas y no contenedores de la aplicación?

Docker no se usa para empaquetar la app sino para aprovisionar dependencias externas reales bajo demanda con Testcontainers. En ese caso levanta una base de datos efímera de mongodb que vive únicamente mientras corre la prueba de integración y se autodestruye al terminar sin requerir instalar nada localmente.

Captura N.º 2: salida de npm install mostrando los paquetes instalados y el archivo package.json creado.

```
kayser@pop-os:~/Documents/NEXUS/UNIVERSIDAD/Octavo_Semestre/com600_microservicios/laboratorios/07-pruebas-automatizadas/practica7-pruebas$ npm init -y
Wrote to /home/kayser/Documents/NEXUS/UNIVERSIDAD/Octavo_Semestre/com600_microservicios/laboratorios/07-pruebas-automatizadas/practica7-pruebas/package.json:

{
  "name": "practica7-pruebas",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}

kayser@pop-os:~/Documents/NEXUS/UNIVERSIDAD/Octavo_Semestre/com600_microservicios/laboratorios/07-pruebas-automatizadas/practica7-pruebas$ npm install express mongodb

added 80 packages, and audited 81 packages in 11s

28 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
kayser@pop-os:~/Documents/NEXUS/UNIVERSIDAD/Octavo_Semestre/com600_microservicios/laboratorios/07-pruebas-automatizadas/practica7-pruebas$ npm install --save-dev jest supertest
npm warn deprecated glob@10.5.0: Old versions of glob are not supported, and contain widely publicized security vulnerabilities, which have been fixed in the current version. Please update. Support for old versions may be purchased (at exorbitant rates) by contacting i@izs.me

added 296 packages, and audited 377 packages in 58s

80 packages are looking for funding
  run `npm fund` for details

19 moderate severity vulnerabilities

To address issues that do not require attention, run:
  npm audit fix

To address all issues (including breaking changes), run:
  npm audit fix --force
```

Laboratorio 1 — La primera prueba unitaria

Captura N.º 3: salida de npm test con la primera prueba pasando en verde.

```
kayser@pop-os:~/Documents/NEXUS/UNIVERSIDAD/Octavo_Semestre/com600_microservicios/laboratorios/07-pruebas-automatizadas/practica7-pruebas$ nano tarea.test.js
kayser@pop-os:~/Documents/NEXUS/UNIVERSIDAD/Octavo_Semestre/com600_microservicios/laboratorios/07-pruebas-automatizadas/practica7-pruebas$ npm test

> practica7-pruebas@1.0.0 test
> jest --runInBand

PASS tests/tarea.test.js
  estadoDeTarea
    ✓ devuelve COMPLETADA cuando la tarea está terminada (2 ms)

Test Suites: 1 passed, 1 total
Tests: 1 passed, 1 total
Snapshots: 0 total
Time: 0.251 s, estimated 1 s
Ran all test suites.
kayser@pop-os:~/Documents/NEXUS/UNIVERSIDAD/Octavo_Semestre/com600_microservicios/laboratorios/07-pruebas-automatizadas/practica7-pruebas$
```

Pregunta N.º 2. El nombre de la prueba es «devuelve COMPLETADA cuando la tarea está terminada» y no «test1». ¿Qué gana el equipo cuando esa prueba falla dentro de seis meses?

Gana diagnostico inmediato y claridad, cuando la prueba falla, el reporte de la terminal describe que regla de negocio se rompió y que resultado se esperaba permitiendo corregir el error de inmediato sin tener que perder el tiempo abriendo e interpretando el código de test

Laboratorio 2 — Casos borde y errores: donde viven los defectos

Captura N.º 4: salida de npm test con las cuatro pruebas del bloque en verde.

```
atizadas/practica7-pruebas/tests$ npm test

> practica7-pruebas@1.0.0 test
> jest --runInBand

PASS tests/tarea.test.js
  ✓ devuelve ATRASADA cuando venció y no está completada
  ✓ una tarea completada nunca figura como atrasada (1 ms)
  ✓ lanza un error si la tarea no tiene título (6 ms)
  estadoDeTarea
    ✓ devuelve COMPLETADA cuando la tarea está terminada (2 ms)

Test Suites: 1 passed, 1 total
Tests:       4 passed, 4 total
Snapshots:   0 total
Time:        0.381 s, estimated 1 s
Ran all test suites.
kayser@pop-os:~/Documents/NEXUS/UNIVERSIDAD/Octavo_Semestre/com600_microservicios/laboratorios/07-pruebas-autom
atizadas/practica7-pruebas/tests$
```

Captura N.º 5: salida de la prueba en rojo, mostrando el valor esperado y el recibido.

```
kayser@pop-os:~/Documents/NEXUS/UNIVERSIDAD/Octavo_Semestre/com600_microservicios/laboratorios/07-pruebas-automatizadas/practica7-pruebas/src$ npm test

> practica7-pruebas@1.0.0 test
> jest --runInBand

FAIL tests/tarea.test.js
  estadoDeTarea
    ✓ devuelve COMPLETADA cuando la tarea está terminada (3 ms)
    ✗ devuelve ATRASADA cuando venció y no está completada (4 ms)
    ✓ una tarea completada nunca figura como atrasada (1 ms)
    ✓ lanza un error si la tarea no tiene título (2 ms)

  • estadoDeTarea › devuelve ATRASADA cuando venció y no está completada

    expect(received).toBe(expected) // Object.is equality

    Expected: "ATRASADA"
    Received: "PENDIENTE"

      10 |   test("devuelve ATRASADA cuando venció y no está completada", () => {
      11 |     const tarea = { titulo: "Pagar servidor", vence: "2026-01-10" };
    > 12 |     expect(estadoDeTarea(tarea, new Date("2026-01-11"))).toBe("ATRASADA");
        |                                                           ^
      13 |   });
      14 |
      15 |   test("una tarea completada nunca figura como atrasada", () => {

      at Object.toBe (tests/tarea.test.js:12:58)

Test Suites: 1 failed, 1 total
Tests:       1 failed, 3 passed, 4 total
Snapshots:  0 total
Time:        0.302 s, estimated 1 s
Ran all test suites.
kayser@pop-os:~/Documents/NEXUS/UNIVERSIDAD/Octavo_Semestre/com600_microservicios/laboratorios/07-pruebas-automatizadas/practica7-pruebas/src$
```

Pregunta N.º 3. Una prueba que nunca ha fallado no demuestra nada. ¿Cómo puede comprobar que una prueba realmente verifica lo que dice verificar?

Modificando deliberadamente la lógica de negocio para forzar a que una prueba falle, si la prueba no falla ante un evidente cambio del código la prueba está mal planteada o no está verificando nada.

Laboratorio 3 — Cobertura: qué mide y qué no

Captura N.º 6: tabla de cobertura impresa en la terminal.

```
kayser@pop-os:~/Documents/NEXUS/UNIVERSIDAD/Octavo_Semestre/com600_microservicios/laboratorios/07-pruebas-automatizadas/practica7-pruebas/src$ npm run test:cov

> practica7-pruebas@1.0.0 test:cov
> jest --coverage

PASS tests/tarea.test.js
  estadoDeTarea
    ✓ devuelve COMPLETADA cuando la tarea está terminada (3 ms)
    ✓ devuelve ATRASADA cuando venció y no está completada (1 ms)
    ✓ una tarea completada nunca figura como atrasada (1 ms)
    ✓ lanza un error si la tarea no tiene título (3 ms)

-----|-----|-----|-----|-----|-----
File    | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-----|-----|-----|-----|-----|-----
All files | 87.5    | 90.9     | 100     | 83.33   |
 tarea.js | 87.5    | 90.9     | 100     | 83.33   | 7
-----|-----|-----|-----|-----|-----
Test Suites: 1 passed, 1 total
Tests:       4 passed, 4 total
Snapshots:   0 total
Time:        0.464 s, estimated 1 s
Ran all test suites.
kayser@pop-os:~/Documents/NEXUS/UNIVERSIDAD/Octavo_Semestre/com600_microservicios/laboratorios/07-pruebas-automatizadas/practica7-pruebas/src$
```

Captura N.º 7: reporte HTML de cobertura mostrando el archivo src/tarea.js con sus líneas coloreadas.

All files tarea.js

87.5% Statements 7/8 90.9% Branches 10/11 100% Functions 1/1 83.33% Lines 5/6

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

```
1 function estadoDeTarea(tarea, ahora = new Date()) {  
2   4x if (!tarea || !tarea.titulo) {  
3   1x   throw new Error("La tarea necesita un titulo");  
4   }  
5   3x   if (tarea.completada) return "COMPLETADA";  
6   1x   E if (tarea.vence && new Date(tarea.vence) < ahora) return "ATRASADA";  
7   return "PENDIENTE";  
8 }  
9  
10  
11 1x module.exports = { estadoDeTarea };  
12
```

Pregunta N.º 4. Escriba una prueba que ejecute todas las líneas de `estadoDeTarea` pero que no verifique nada útil (por ejemplo, sin ningún `expect` que compare el resultado). ¿Qué cobertura reporta? ¿Qué conclusión saca sobre la cobertura como criterio de calidad?

Reporta 100 por ciento de cobertura de líneas, la conclusión es que la cobertura es una métrica de ejecución pero no mide la calidad de las acepciones

Laboratorio 4 — Probar el endpoint HTTP sin levantar el servidor

Captura N.º 8: salida de `npm test` con las pruebas de la API en verde.

```
kayserpop-os: ~/Documents/NEXUS/UNIVERSIDAD/Octavo_Semestre/com600_microservicios/laboratorios/07-pruebas
-automatizadas/practica7-pruebas$ npm test -- --verbose

> practica7-pruebas@1.0.0 test
> jest --runInBand --verbose

PASS tests/tarea.test.js
  estadoDeTarea
    ✓ devuelve COMPLETADA cuando la tarea está terminada (3 ms)
    ✓ devuelve ATRASADA cuando venció y no está completada
    ✓ una tarea completada nunca figura como atrasada (1 ms)
    ✓ lanza un error si la tarea no tiene título (9 ms)

PASS tests/api.test.js
  ✓ POST /tareas crea la tarea y responde 201 (33 ms)
  ✓ POST /tareas sin título responde 400 y no crea nada (7 ms)

Test Suites: 2 passed, 2 total
Tests:       6 passed, 6 total
Snapshots:   0 total
Time:        0.379 s, estimated 1 s
Ran all test suites.
```

Captura N.º 9: salida completa de la suite con el tiempo total de ejecución visible.

```
kayserpop-os: ~/Documents/NEXUS/UNIVERSIDAD/Octavo_Semestre/com600_microservicios/laboratorios/07-pruebas
-automatizadas/practica7-pruebas$ npm test -- --verbose

> practica7-pruebas@1.0.0 test
> jest --runInBand --verbose

PASS tests/tarea.test.js
  estadoDeTarea
    ✓ devuelve COMPLETADA cuando la tarea está terminada (3 ms)
    ✓ devuelve ATRASADA cuando venció y no está completada
    ✓ una tarea completada nunca figura como atrasada (1 ms)
    ✓ lanza un error si la tarea no tiene título (9 ms)

PASS tests/api.test.js
  ✓ POST /tareas crea la tarea y responde 201 (33 ms)
  ✓ POST /tareas sin título responde 400 y no crea nada (7 ms)

Test Suites: 2 passed, 2 total
Tests:       6 passed, 6 total
Snapshots:   0 total
Time:        0.379 s, estimated 1 s
Ran all test suites.
```

Laboratorio 5 — Integración real con un contenedor efímero

Captura N.º 10: las dos terminales: la suite ejecutándose y docker ps mostrando el contenedor efímero levantado por la prueba.

```
kayser@pop-os:~/Documents/NEXUS/UNIVERSIDAD/Octavo_Semestre/com600_microservicios/laboratorios/07-pruebas
-automatizadas/practica7-pruebas$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS
a921774d7c69   mongo:7                             "docker-entrypoint.s..." 2 seconds ago  Up 1 second (
health: starting) 0.0.0.0:32785->27017/tcp, [::]:32785->27017/tcp   elegant_cohen
55c5c24f202f   testcontainers/ryuk:0.14.0         "/bin/ryuk"             3 seconds ago  Up 2 seconds
0.0.0.0:32784->8080/tcp, [::]:32784->8080/tcp      testcontainers-ryuk-ffa182ed5f22
fb8ba4ae19bd   moodle-sandbox-moodle-sandbox-app  "httpd -D FOREGROUND"    8 days ago    Up 4 hours
0.0.0.0:8080->80/tcp, [::]:8080->80/tcp            moodle_sandbox_web
bcbdf3d2ee1c   postgres:17-alpine                "docker-entrypoint.s..." 7 weeks ago   Up 4 hours (h
ealthy)         0.0.0.0:5433->5432/tcp, [::]:5433->5432/tcp      bd_postgres
kayser@pop-os:~/Documents/NEXUS/UNIVERSIDAD/Octavo_Semestre/com600_microservicios/laboratorios/07-pruebas
-automatizadas/practica7-pruebas$
```

```
File Edit View kayser@pop-os: ~/Documents/NEXUS/UNIVERSIDAD/Octavo_Semestre/com600_m... + - .
(node:75346) ExperimentalWarning: VM Modules is an experimental feature and might change at any time
(Use 'node --trace-warnings ...' to show where the warning was created)
PASS tests/repositorio.int.test.js (6.713 s)
PASS tests/api.test.js
PASS tests/tarea.test.js

Test Suites: 3 passed, 3 total
Tests:       7 passed, 7 total
Snapshots:   0 total
Time:        7.124 s, estimated 8 s
Ran all test suites.
kayser@pop-os:~/Documents/NEXUS/UNIVERSIDAD/Octavo_Semestre/com600_microservicios/laboratorios/07-pruebas
-automatizadas/practica7-pruebas$ npm test

> practica7-pruebas@1.0.0 test
> node --experimental-vm-modules node_modules/jest/bin/jest.js --runInBand

RUNS tests/repositorio.int.test.js
```

Captura N.º 11: salida mostrando que el contenedor de la prueba ya no existe después de terminar la suite.

```
kayser@pop-os:~/Documents/NEXUS/UNIVERSIDAD/Octavo_Semestre/com600_microservicios/laboratorios/07-pruebas-automatizadas/practica7-pruebas$ docker ps | grep mongo
kayser@pop-os:~/Documents/NEXUS/UNIVERSIDAD/Octavo_Semestre/com600_microservicios/laboratorios/07-pruebas-automatizadas/practica7-pruebas$
```

Pregunta N.º 5. ¿Qué problema concreto evita este enfoque frente a la alternativa de que cada estudiante instale MongoDB en su propia máquina y las pruebas apunten a esa instalación?

Evita inconsistencias de entorno, diferencias de versiones y colisiones de datos, garantiza que la prueba siempre se ejecute contra una base limpia, aislada e idéntica funcionando de la misma manera en la máquina de un estudiante o en la de un servidor limpio de CI sin riesgo de alterar datos locales

Laboratorio 6 — Prueba de contrato entre dos servicios

Captura N.º 12: la prueba de contrato en verde junto al resto de la suite.

```
kayser@pop-os:~/Documents/NEXUS/UNIVERSIDAD/Octavo_Semestre/com600_microservicios/laboratorios/07-pruebas-automatizadas/practica7-pruebas$ npm test

> practica7-pruebas@1.0.0 test
> node --experimental-vm-modules node_modules/jest/bin/jest.js --runInBand

(node:77767) ExperimentalWarning: VM Modules is an experimental feature and might change at any time
(Use `node --trace-warnings ...` to show where the warning was created)
PASS tests/repositorio.int.test.js (6.735 s)
PASS tests/contrato.test.js
PASS tests/api.test.js
PASS tests/tarea.test.js

Test Suites: 4 passed, 4 total
Tests:      8 passed, 8 total
Snapshots:  0 total
Time:       7.201 s, estimated 8 s
Ran all test suites.
kayser@pop-os:~/Documents/NEXUS/UNIVERSIDAD/Octavo_Semestre/com600_microservicios/laboratorios/07-pruebas-automatizadas/practica7-pruebas$
```

Captura N.º 13: la prueba de contrato fallando y señalando el campo faltante.

```
kayser@pop-os:~/Documents/NEXUS/UNIVERSIDAD/Octavo_Semestre/com600_microservicios/laboratorios/07-pruebas-automatizadas/practica7-pruebas$ npm test

> practica7-pruebas@1.0.0 test
> node --experimental-vm-modules node_modules/jest/bin/jest.js --runInBand

FAIL tests/contrato.test.js
  • el evento publicado cumple el contrato acordado

    expect(received).toHaveProperty(path)

    Expected path: "creadaEn"
    Received path: []

    Received value: {"fecha": "2026-10-08T17:33:14.267Z", "id": 1, "titulo": "Estudiar"}

       5 |   const evento = construirEventoTareaCreada({ id: 1, titulo: "Estudiar" });
       6 |   for (const campo of contrato.camposObligatorios) {
     >  7 |     expect(evento).toHaveProperty(campo);
         |                       ^
       8 |     expect(typeof evento[campo]).toBe(contrato.tipos[campo]);
       9 |   }
      10 | });

at Object.toHaveProperty (tests/contrato.test.js:7:20)

RUNS tests/repositorio.int.test.js
```

Pregunta N.º 6. Las pruebas unitarias del servicio de Tareas siguieron en verde con ese cambio. ¿Por qué? ¿Qué tipo de defecto detecta la prueba de contrato que ningún otro nivel detecta?

siguieron en verde porque las unitarias solo validan que la función interna produzca un objeto válido y no alcance excepciones locales, la prueba de contrato detecta rupturas en la interfaz de comunicación entre servicios que harían fallar al consumidor en producción aunque el emisor funcione bien por su cuenta

Laboratorio 7 — Dejar el proyecto listo para el pipeline

Captura N.º 14: los dos códigos de salida: 0 con la suite en verde y distinto de 0 con la suite en rojo.

```
bas$ npm test
echo $?

> practica7-pruebas@1.0.0 test
> node --experimental-vm-modules node_modules/jest/bin/jest.js --runInBand

PASS tests/contrato.test.js
(node:80660) ExperimentalWarning: VM Modules is an experimental feature and might change at any time
(Use `node --trace-warnings ...` to show where the warning was created)
PASS tests/repositorio.int.test.js (6.774 s)
PASS tests/api.test.js
PASS tests/tarea.test.js

Test Suites: 4 passed, 4 total
Tests: 8 passed, 8 total
Snapshots: 0 total
Time: 7.419 s, estimated 8 s
Ran all test suites.
0

keyser@pop-os:~/Documents/NEXUS/UNIVERSIDAD/Octavo_Semestre/com600_microservicios/laboratorios/07-pruebas-automatizadas/practica7-prue
bas$ sed -i 's/creadaEn/fecha/' src/eventos.js && npm run test:unit; echo "Codigo de salida en rojo:
$?"; sed -i 's/fecha/creadaEn/' src/eventos.js

> practica7-pruebas@1.0.0 test:unit
> node --experimental-vm-modules node_modules/jest/bin/jest.js --testPathIgnorePatterns int.test

PASS tests/api.test.js
FAIL tests/contrato.test.js
  • el evento publicado cumple el contrato acordado

    expect(received).toHaveProperty(path)

    Expected path: "creadaEn"
    Received path: []

    Received value: {"fecha": "2026-10-08T17:38:43.451Z", "id": 1, "titulo": "Estudiar"}

       5 |     const evento = construirEventoTareaCreada({ id: 1, titulo: "Estudiar" });
       6 |     for (const campo of contrato.camposObligatorios) {
     >    7 |       expect(evento).toHaveProperty(campo);
         |                       ^
       8 |       expect(typeof evento[campo]).toBe(contrato.tipos[campo]);
       9 |     }
      10 |   });

at Object.toHaveProperty (tests/contrato.test.js:7:20)

PASS tests/tarea.test.js

Test Suites: 1 failed, 2 passed, 3 total
Tests: 1 failed, 6 passed, 7 total
Snapshots: 0 total
Time: 0.528 s, estimated 1 s
Ran all test suites.
Codigo de salida en rojo:
1
```

Pregunta N.º 7. Si una prueba es inestable (a veces pasa y a veces falla), ¿qué le ocurre al pipeline y qué le ocurre a la confianza del equipo? ¿Qué haría usted con esa prueba?

Genera una prueba intermitente lo que provoca falsos positivos detiene despliegues validos y hace que el equipo pierda confianza en el pipeline hasta acostumbrarse a ignorar los fallos, lo que se debe hacer es poner la prueba en cuarentena encontrar la causa raiz y corregirla antes de reactivarla

Captura N.º 15: la página del repositorio en GitHub mostrando la carpeta practica7-pruebas con tests/, contratos/ y el README.



Cierre de la Parte 1

Responda brevemente antes de entregar:

¿Qué parte de la práctica le resultó más difícil y por qué?

la parte de los contenedores de inicio no me salia vacio pero era porque yo tenia contenedores antiguos de mongo que pensaba eran los de la practica y me dificulto

¿Qué error tuvo que resolver por su cuenta y cómo lo resolvió?

Ninguno, en esta practica igual que la anterior el avance fue fluido y no habia nada que no estaba en la guía

¿Qué haría distinto si tuviera que repetir esta práctica?

Por esta vez siento que hice todo de la manera mas eficiente que podia

Declaración de autoría: Declaro que las capturas presentadas fueron tomadas en mi propio equipo y que las respuestas son de mi autoría. Entiendo que presentar evidencias de otra persona anula la Parte 1 de esta práctica.

_____valencia medina freddy
daniel_____
Firma del estudiante

Fecha